SOURCE-LEVEL GUIDE · RSSHUB × FOLO

RSSHub + Folo：从原理到源码

一份为“自己动手 contribute / 定制 RSS 阅读器”准备的导读：两个项目各自要解决什么问题、有哪些限制、架构如何拆分、组件之间怎样连接，以及开源许可证对你意味着什么。

DIYgod/RSSHub @ 8f4fd43 · RSSNext/Folo @ 158ecbe

怎么读这份导读

这份导读给想做三件事的人：**读懂** RSSHub 和 Folo 的工作原理；**给它们提 PR**；**在它们之上定制出自己喜欢的 RSS 阅读器**。

全文按这个顺序展开：

1. **全景**：RSS 生态本来有什么问题，两个项目各自解决哪一段，它们之间靠什么连起来。
2. **RSSHub**：一个请求从进入到吐出 RSS 的完整路径，路由、缓存、抓取、输出、部署。
3. **Folo**：monorepo 的结构、离线优先的数据层、同步引擎、Electron 架构、AI 功能。
4. **端到端**：把 `rsshub://hackernews` 从“订阅”到“显示在时间线上”串一遍。
5. **开源许可证**：AGPL-3.0 对“自己用、提 PR、发 fork、发安装包、对外提供服务”分别意味着什么。
6. **贡献与定制**：开发环境、提交规范、适合入手的方向，以及“打造自己的阅读器”的三条路线。

每张图都是可交互的 React Flow 风格节点图：在 HTML 里把鼠标悬停在节点上，会高亮它的所有连接。正文里蓝底的链接都指向 GitHub 上**固定 commit** 的源码行，所以即使上游继续改动，链接也不会漂移。

> **要点 · 两个版本**
>
> - RSSHub：`DIYgod/RSSHub` @ `8f4fd43`（2026-11-05）
> - Folo：`RSSNext/Folo` @ `158ecbe`

全景：RSS 生态的问题与两个项目的分工

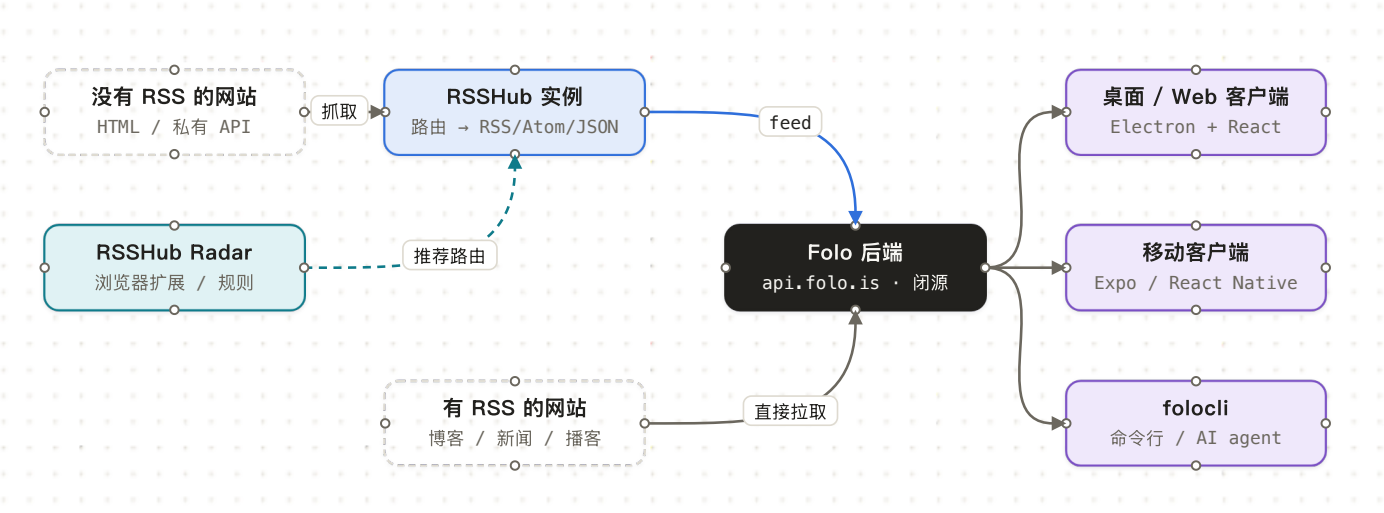
RSS 本来要解决什么，现在又卡在哪里

RSS/Atom 的设想很简单：网站发布一份机器可读的“更新列表”，阅读器定时拉取，用户在一个地方看所有来源的更新，不依赖算法推荐、不需要逐个打开网站。

现实中它卡在三处：

环节	问题	谁来解决
来源	大量网站（社交媒体、视频站、论坛、政府网站、App 内容）根本不提供 RSS，或者只提供残缺的摘要	**RSSHub**：为每个网站写一个“路由”，把网页 / API 转成标准 feed
抓取与同步	每个阅读器都要自己定时抓取成千上万个 feed，还要在多设备之间同步“已读、收藏、分类”	**Folo 后端**（闭源服务，`api.folo.is`）：集中抓取、去重、存储，并把用户状态同步到所有客户端
阅读体验	传统阅读器把所有内容都当成“文章列表”，看图片、视频、播客、社交动态都很别扭；信息太多读不完	**Folo 客户端**（开源）：按内容类型分成 6 种视图，加上 AI 摘要、翻译、排序和对话

全景：数据从网站流到你的屏幕



蓝色是开源的 RSSHub，紫色是开源的 Folo 客户端；黑色的 Folo 后端不在公开仓库里，客户端通过 HTTP API 和它通信。

两个项目的“要求、目标、限制”

理解一个系统最快的方法，是先弄清它在什么约束下做取舍。下面这张表是整篇导读的骨架，后面每一节都在回答“它是怎么在这些限制下做到的”。

	RSSHub	Folo
核心目标	“万物皆可 RSS”：任何网站都能用一个 URL 订阅	一个现代的、AI 加持的 RSS 阅读器，多端一致
功能要求	几千条路由，每条把一个网站转成标准 feed；支持过滤、全文、多种输出格式	订阅任意 RSS / Atom / JSON feed 和 RSSHub 路由；6 种内容视图；多设备同步；离线可读；AI
规模压力	2000 个命名空间、上万条路由，由几千名社区贡献者维护；一个热门实例每天被阅读器反复请求	用户的时间线可能有上万条未读；首屏要快；手机、电脑、Web 都要能用
外部限制	目标网站随时改版、反爬、限流、需要登录；抓取慢（可能要开浏览器）	依赖自家后端；Electron / Web / iOS / Android 四种运行时的差异；App Store 规则
关键解决手段	文件即路由 + 构建期生成路由表 + 懒加载；两级缓存 + 防击穿锁；统一的数据契约	共享的 `packages/internal` 业务层；SQLite 本地库 + 内存 store；Linear 式增量同步；按视图分模板
开源许可证	AGPL-3.0	AGPL-3.0-only（附加一条图标例外）

类比 · 翻译社与报刊亭

RSSHub 像一家“翻译社”：每个网站说自己的方言，RSSHub 为每种方言雇一个译员（路由），全部翻译成标准普通话（RSS）。Folo 像一家“报刊亭 + 私人助理”：后端负责每天去各家报社取报纸，客户端负责把报纸、画报、录音带分类摆好，并且可以替你先读一遍、划重点。

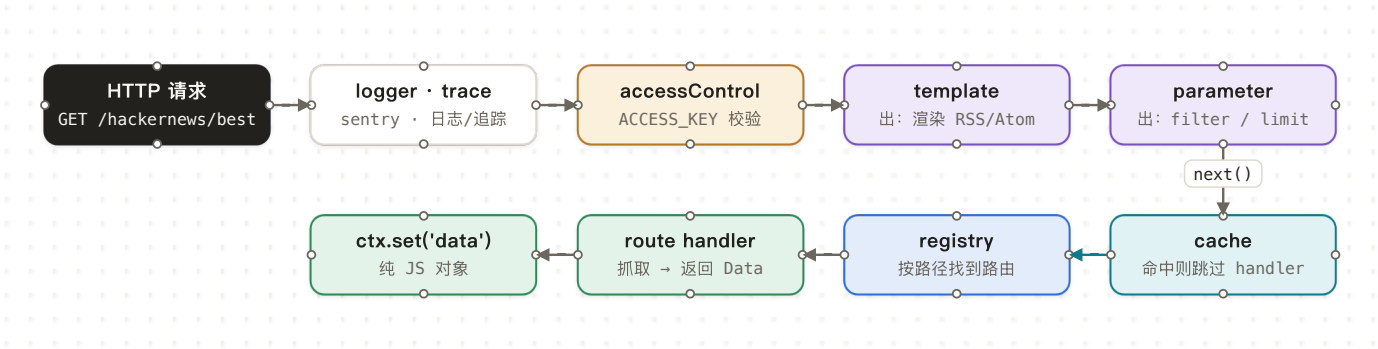
RSSHub：把任何网站变成 feed

一个请求的完整路径

RSSHub 是一个 Hono Web 服务。启动入口 `lib/index.ts` 用 `@hono/node-server` 监听端口（默认 1200，见 `config.ts`），可选 cluster 多进程。真正的应用组装在 `lib/app-bootstrap.tsx`：一串中间件，最后挂上路由表。

Hono 的中间件是“洋葱模型”：每个中间件先做“进去时”的事，调用 `await next()` 进入下一层，等内层返回后再做“出来时”的事。RSSHub 巧妙地利用了这一点：**路由 handler 只返回一个 JS 对象（`Data`），过滤、改写、渲染成 XML 全部发生在“出来的路上”。**

洋葱模型：进去时检查，出来时加工

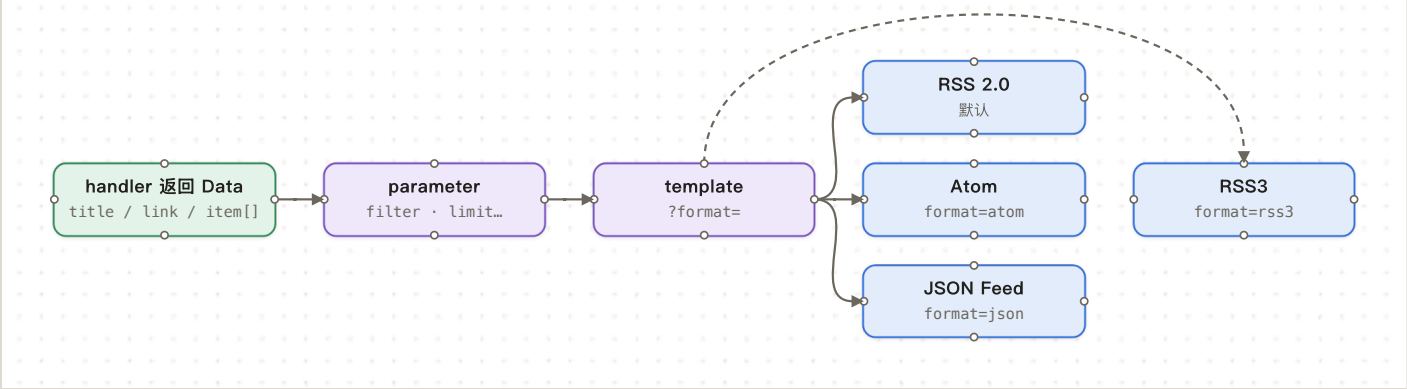


上排是“进去时”的顺序；handler 写入 `ctx.data` 后，控制权按相反顺序回到 cache（写缓存）→ parameter（过滤）→ antiHotlink（改写图片）→ template（序列化）。中间件注册顺序见 `app-bootstrap.tsx`。

几个值得记住的细节：

- **handler 不碰输出格式。** `template.tsx` 在 `await next()` 之后读取 `ctx.get('data')`，再根据 `?format=` 选择输出（`rss / atom / json / rss3`）。所以你写路由时只需要关心“这个网站有哪些条目”，同一条路由自动支持所有格式。
- **过滤是通用能力。** `parameter.ts` 同样在 `next()` 之后工作：`?filter=` 正则过滤（`L239`）、`?filterout=`、`?limit=`、`?mode=fulltext` 全文抽取、`?opencc=` 简繁转换、`?brief=` 截断等。这些参数对所有几千条路由都生效，路由作者不用自己实现。
- **错误也是统一的。** handler 抛出的 `InvalidParameterError` 等错误由 `app.onError` 统一渲染（`L52`）。

同一份 Data，多种输出



例：`/hackernews/best?filter=AI|GPU&limit=10&format=atom`，路由代码完全不用管这些参数。渲染组件在 `lib/views/`。

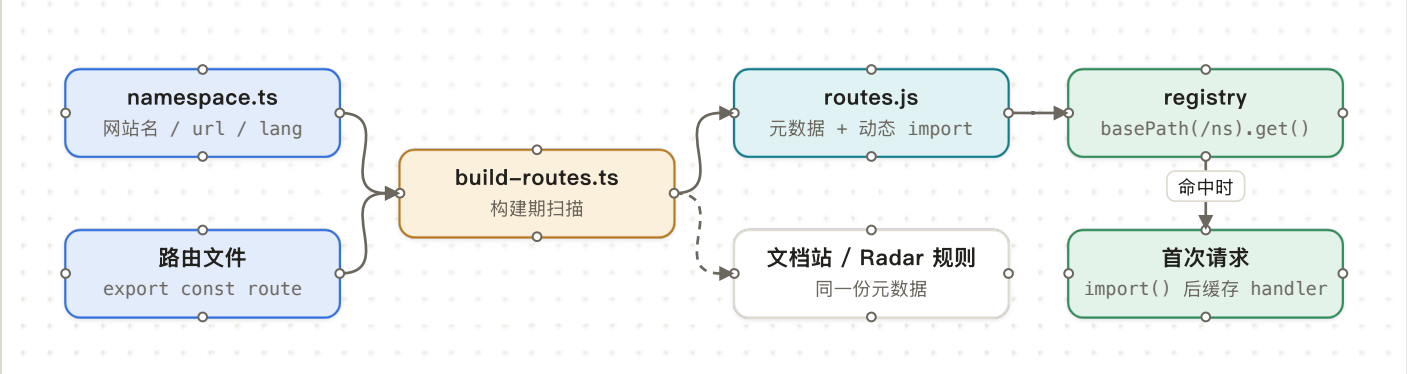
路由系统：文件即路由，构建期生成路由表

RSSHub 的路由按**命名空间**组织：每个网站一个目录 `lib/routes/<namespace>/`，目录里一个 `namespace.ts` 描述网站本身，若干个路由文件各导出一个 `route` 对象。当前快照里有约 2000 个命名空间。

问题来了：几千个路由文件，如果启动时全部 `import`，启动会极慢、内存会很大。RSSHub 的解法分两步：

1. **构建期扫描。** `scripts/workflow/build-routes.ts` 遍历所有路由文件，提取每条路由的元数据（path、名称、参数、radar 规则、分类……），生成 `assets/build/routes.js`。这份文件同时也是文档站和 Radar 规则的数据来源。
2. **运行期懒加载。** `registry.ts` 读入路由表，`registerRssRoutes` 为每个命名空间创建 `app.basePath('/<namespace>')`，为每条路由注册一个包装 handler；**直到第一次有请求命中，才真正 `import` 对应的路由模块**（`L194`），之后缓存在 `routeData.handler` 上。

路由从源码到可访问



你新加一个路由文件，`pnpm dev` 会重新生成路由表；开发模式下还有一个 dev registry 按需扫描目录（`registry.ts`）。

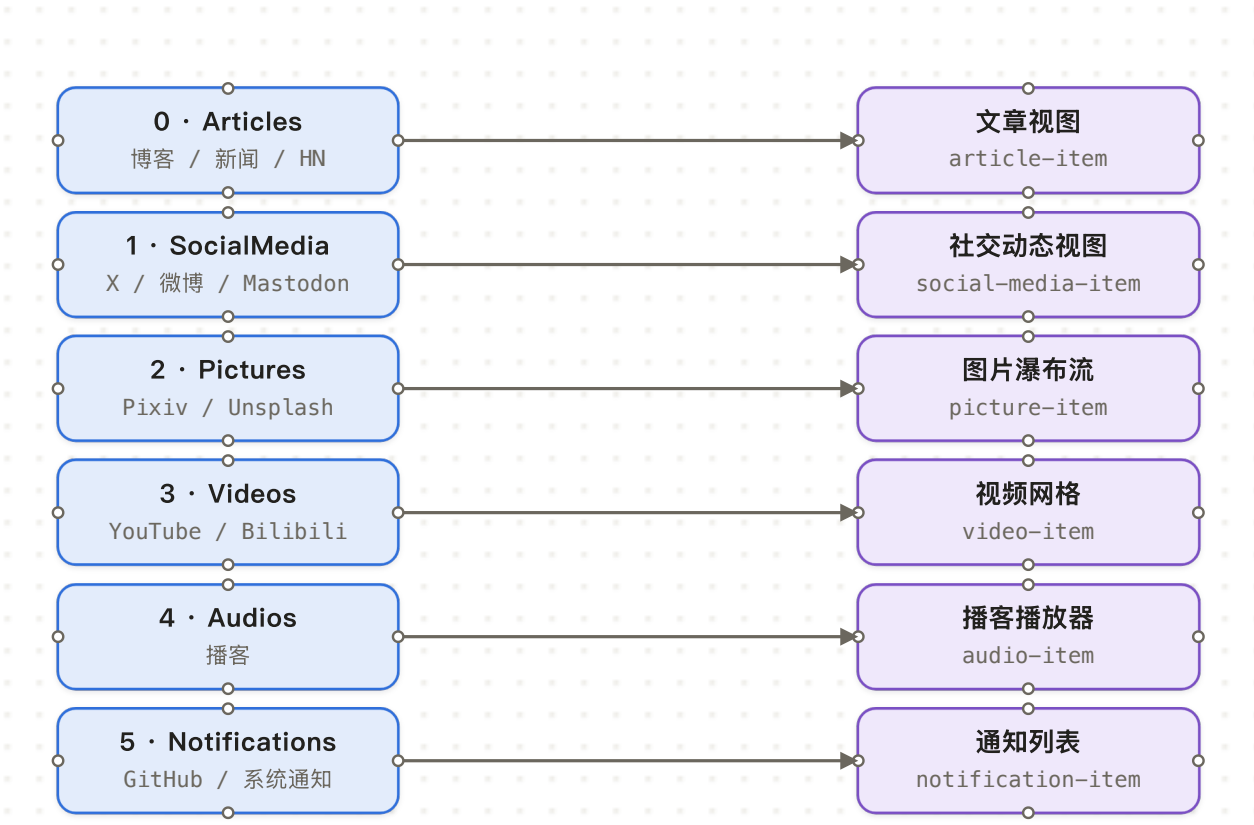
路由的数据契约：Route、Data、DataItem

路由作者和框架之间只靠三个类型交流，全部定义在 `lib/types.ts`：

类型	作用	关键字段
`Route`	描述“这条路由是什么”	`path`（Hono 路由语法，如 `/:section?/:type?`）、`name`、`example`、`parameters`、`categories`、`features`（是否需要配置、是否要浏览器、是否反爬）、`radar`、`maintainers`、`handler`、`view`
`Data`	handler 的返回值，即整份 feed	`title`、`link`、`description`、`image`、`item[]`、`language`、`ttl`
`DataItem`	feed 里的一条	`title`、`link`、`guid`、`description`（HTML）、`pubDate`、`author`、`category`、`enclosure_url`（播客 / BT）、`media`

注意 `Route.view` 用的 `ViewType` 枚举：`Articles=0, SocialMedia=1, Pictures=2, Videos=3, Audios=4, Notifications=5`。**这正是 Folo 的 6 种视图**，RSSHub 的路由作者在声明路由时就告诉了 Folo“这个源应该用哪种界面展示”。两个项目的耦合点之一就在这里。

ViewType：RSSHub 路由声明 → Folo 视图



左边是 RSSHub 的 `ViewType`，右边是 Folo 的列表项组件（`entry-column/Items/`）。编号完全一致。

一个真实的例子：`hackernews 路由`。它的 `path` 是 `/:section?/:type?/:value?/:minComments?`，所以 `/hackernews`、`/hackernews/best`、`/hackernews/over/sources/100` 都由它处理；`radar` 字段（`L41`）声明了“在 news.ycombinator.com/best 页面上，推荐订阅 /hackernews/best”。

抓取：列表页 + 详情页 + 缓存

几乎所有路由都遵循同一个套路，`hackernews 的 handler` 是典型：

```
// 简化自 lib/routes/hackernews/index.ts
async function handler(ctx) {
    const section = ctx.req.param('section') ?? 'index';
    const response = await got(currentUrl);          // 1. 抓列表页
    const $ = load(response.data);                     // 2. cheerio 解析 HTML
    const list = $('.athing').toArray().map((thing) => ({
        title: $(thing).find('.titleline').children('a').text(),
        link: `${rootUrl}/item?id=${$(thing).attr('id')}`,
        description: `<a href="...">Comments on Hacker News</a> | <a href="...">Source</a>
    })).slice(0, getRouteLimit(ctx, 30));             // 3. 尊重 ?limit=

    const items = await pMap(list, (item) =>           // 4. 详情页：并发抓取 + 缓存
        cache.tryGet(`hackernews:comments:${item.link}`, () => fetchDetail(item)));

    return { title: 'Hacker News', link: rootUrl, item: items };   // 5. 返回 Data
}
```

> **和我们之前的实测对上了**
>
> 前面用 folocli 看 `rsshub://hackernews` 时，每篇的描述都只有“Comments on Hacker News | Source”。原因就在上面第 3 步：默认的 `type=sources` 只生成这一行链接，不抓原文。想要评论，可以订阅 `/hackernews/index/comments`；想要原文，可以加 `?mode=fulltext` 让 parameter 中间件去抽取正文。

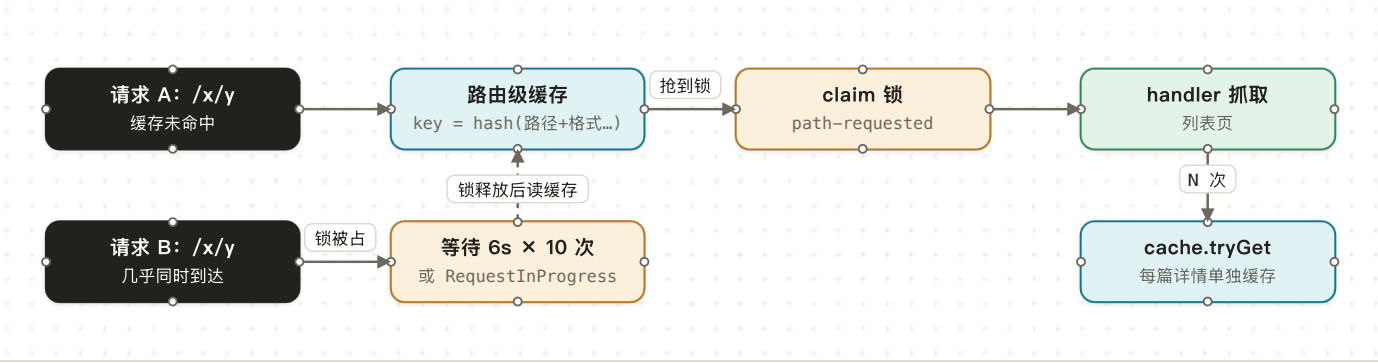
抓取工具都在 `lib/utils/`：

- **HTTP**：`ofetch.ts`（带重试，遇到 429/5xx 自动重试 `REQUEST_RETRY` 次）和兼容旧代码的 `got.ts`。它们统一注入 User-Agent、代理（`PROXY_URI`）、按主机限流（`REQUEST_RATE_LIMITS`）。
- **HTML 解析**：`cheerio` 的 `load()`。
- **浏览器**：反爬严重、必须执行 JS 的网站用 `playwright.ts`（基于 `patchright`，可以连远程浏览器）。这类路由要在 `features.requirePuppeteer` 里声明，因为开浏览器很贵，部分部署形态（如 Workers）不支持。
- **日期**：`parseDate`、`parseDateInTimezone`，因为各网站的时间格式五花八门。

缓存：两级缓存 + 防击穿

RSSHub 最重要的工程设计是缓存。阅读器会以分钟级频率反复请求同一个 feed，而目标网站经不起这样的压力（会封 IP），抓取本身也慢。RSSHub 用了**两级缓存**：

两级缓存与“请求进行中”锁



路由级缓存在 `middleware/cache.ts`，详情级缓存是 `cache.tryGet`。

1. **路由级（整份 feed）**：`cache.ts` 把“路径 + format + limit + sorted + 是否带过滤参数”哈希成 key。命中就直接把缓存的 `Data` 放进 `ctx`，跳过 handler；未命中则执行 handler，再把结果写回（`L93`，`CACHE_EXPIRE` 默认 5 分钟，见 `config.ts`）。
2. **防击穿（cache stampede）**：同一个冷门 feed 被 100 个阅读器同时请求时，不能让 100 个请求都去打目标网站。`L40` 用一个 `claim` 原子锁，只让第一个请求去抓，其他请求每 6 秒检查一次，最多 10 次，超时就返回“This path is currently fetching”。
3. **详情级（单篇文章）**：列表页通常每次都变，但每篇文章的详情页基本不变。`cache.tryGet(key, fn)` 让路由作者把“抓详情页”的结果单独缓存（`CACHE_CONTENT_EXPIRE` 默认 1 小时）。列表刷新时，只有新出现的文章才真正发请求。**这是给 RSSHub 写路由时 review 最常要求的一点。**

缓存后端可选 memory（单进程 LRU）、redis、http、Cloudflare KV，见 `lib/utils/cache/` 和 `CACHE_TYPE`。

> **注意 · 一个容易误解的地方**
>
> 路由级缓存存的是 handler 的**原始输出**，因为 cache 是最内层的中间件，它“出来时”先执行，parameter 的过滤和 template 的渲染在它之后。所以带不同 `?filter=` 的请求可以共享抓取结果；key 里之所以包含 `limit`、`filtered` 这些标志，是因为路由自己也会读它们（比如 `getRouteLimit(ctx)` 决定抓多少条）。

Radar：从“我在看的网页”反推订阅地址

用户通常不知道某个网站对应哪条 RSSHub 路由。每条路由的 `radar` 字段声明了“哪些网页 URL 对应这条路由”，构建期汇总后通过 `/api/radar/rules` 对外提供。

- **RSSHub Radar** 浏览器扩展读取这些规则：你打开 `news.ycombinator.com/best`，它就提示可以订阅 `/hackernews/best`。
- **Folo 的发现页**也用同样的思路：在搜索框里粘贴一个网页 URL，Folo 会尝试匹配出对应的 `rsshub://` 路由（见 `UnifiedDiscoverForm.tsx`、`DiscoverFeedForm.tsx`）。

部署形态与关键配置

同一份代码可以以多种形态运行，构建配置分别是仓库根目录的 `tsdown*.config.ts` 和 `wrangler*.toml`：

形态	入口	适合	限制
Node.js / Docker	`lib/index.ts`	自建实例（最常见）	需要自己维护；开浏览器的路由需要 Chromium
Cloudflare Workers	`lib/worker.ts`	免运维、全球边缘	没有文件系统，私有路由要在构建期打包；浏览器能力受限
Vercel / Railway / Fly	根目录各平台配置	一键部署	受平台执行时间限制

自建实例最常用的环境变量（全部在 `lib/config.ts`）：`PORT`、`CACHE_TYPE` / `REDIS_URL`、`CACHE_EXPIRE`、`ACCESS_KEY`（实例访问控制，见 `access-control.ts`）、`PROXY_URI`、`REQUEST_RATE_LIMITS`、各网站的 Cookie / Token（如 Twitter、Bilibili），以及 `USER_ROUTES_PATH`（挂载你自己的私有路由，见下一节）。

和 Folo 的专属接口

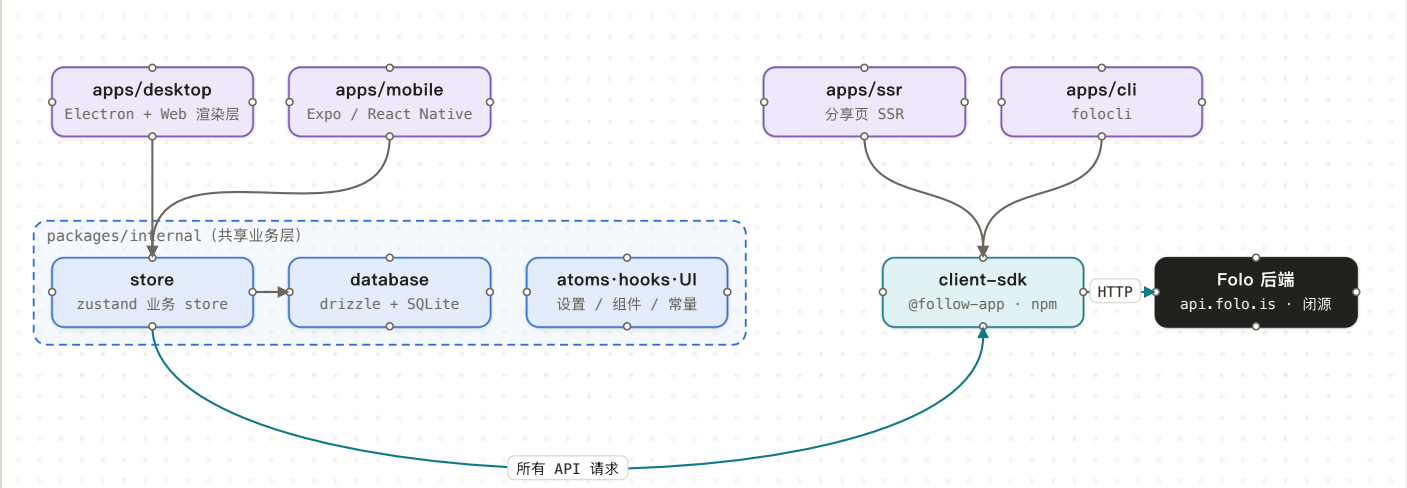
RSSHub 里有一个专门为 Folo 准备的接口：`/api/follow/config`（Folo 的前身叫 Follow）。它返回 `ownerUserId`、`description`、`price`、`userLimit`、`cacheTime`、`gitHash`。这是 Folo 的“**RSSHub 实例市场**”用的：任何人都可以把自己部署的 RSSHub 实例登记到 Folo，设置价格和用户上限，其他 Folo 用户可以选择通过这个实例抓取 `rsshub://` 源。第 4 章会把这条链路完整画出来。

Folo：一个多端的 AI RSS 阅读器

Monorepo 地图

Folo 是一个 pnpm workspace + Turborepo 的 monorepo（见 `AGENTS.md`、`pnpm-workspace.yaml`）。最重要的设计决策是：**业务逻辑放在 `packages/internal`，被桌面、Web、移动端共享；各个 app 只负责自己运行时特有的部分。**

Monorepo：apps 共享 packages/internal



实线箭头是“依赖 / 调用”。`@follow-app/client-sdk` 是单独发布到 npm 的 API 客户端（MIT 许可），类型定义和 `FeedViewType` 都来自它（`enums.ts`）。

目录	是什么	你可能改它来做什么
`apps/desktop/layer/main`	Electron 主进程：窗口、菜单、托盘、更新、深链、IPC 服务	系统集成、快捷键、本地文件能力
`apps/desktop/layer/renderer`	React SPA，同时是桌面端的界面和 `app.folo.is` Web 版	绝大多数 UI / 交互改动
`apps/mobile`	Expo / React Native 应用	移动端
`apps/ssr`	分享链接的服务端渲染（OG 标签等，见 `meta-handler.ts`）	分享页样式
`apps/cli`	`folocli` 命令行和 `skill.md`	脚本化、AI agent
`packages/internal/store`	每种数据一个 store：entry、feed、subscription、unread、summary、translation…	数据逻辑、同步
`packages/internal/database`	drizzle schema、迁移、各平台 SQLite 驱动	本地持久化
`packages/readability`	正文抽取的清洗	阅读模式

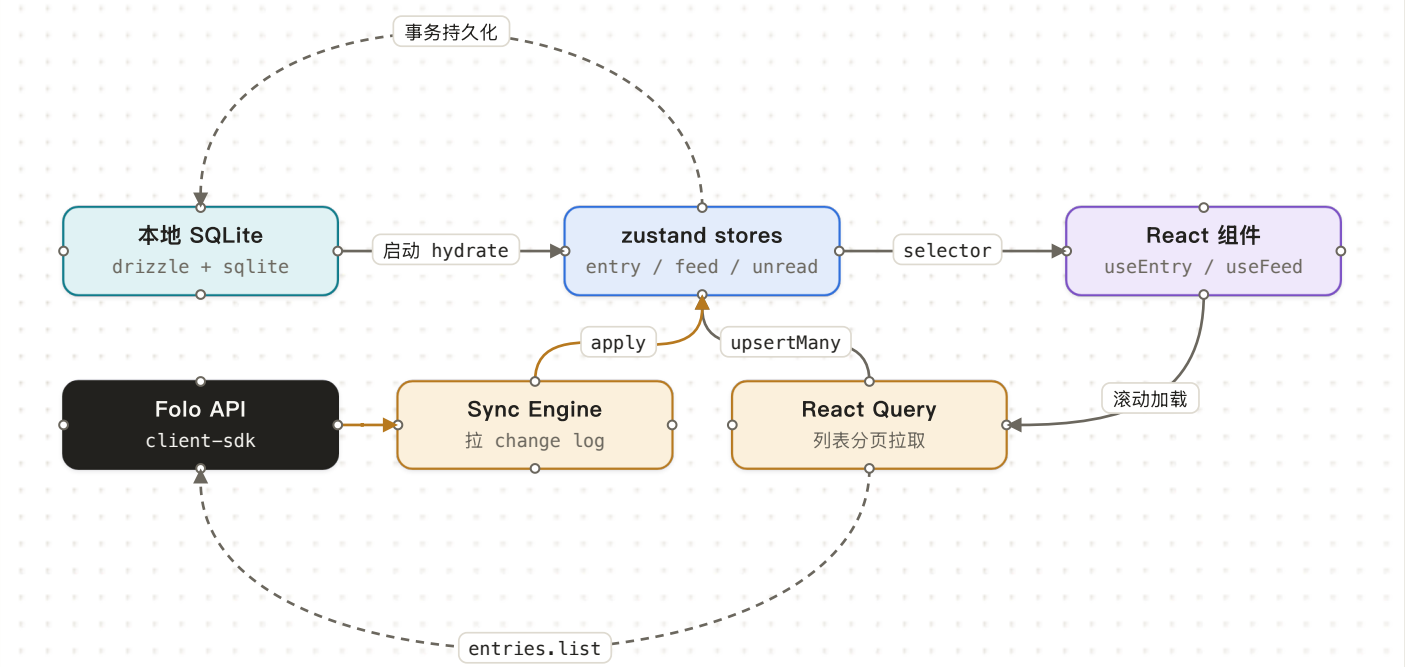
最关键的限制：后端不在仓库里

> **注意 · 先弄清楚这一点，再决定怎么定制**
>
> Folo 的公开仓库**只有客户端**。订阅、抓取 feed、存储条目、未读计数、AI（摘要 / 翻译 / 排序 / 对话）、付费计划，全部由 `api.folo.is` 后端完成；客户端通过 `FollowClient`（来自 `@follow-app/client-sdk`）调用它。这意味着：
>
> - 你可以自由修改和发布客户端，但它仍然连接 Folo 的官方服务，受 Folo 的服务条款约束；
> - 你**无法**仅凭这个仓库搭出一个完全独立、自托管的 Folo；
> - 想改变“服务端行为”（比如抓取频率、AI 模型），要么在客户端绕开（像我们在本地做的“自带 OpenAI key”那样），要么自己另建后端。

离线优先的数据层

Folo 的客户端不是“每次都去问服务器”，而是**本地数据库 + 内存 store + 后台同步**。打开 App 时先从本地 SQLite 把数据灌进内存，界面立刻可用，然后再和服务器对齐。

数据层：SQLite → store → React，网络在旁路



桌面端用 wa-sqlite + IndexedDB 存储（`db.desktop.ts`），移动端用 expo-sqlite（`db.rn.ts`）；两者共用同一套 drizzle schema（`schemas/index.ts`）。

关键代码：

- **启动灌数据**：`initialize/index.ts` 调用 `hydrateDatabaseToStore`，从 SQLite 读出所有表，写进各个 store。
- **store 的形状**：`EntryState` 除了 `data`（id → 条目），还维护了多组索引：`entryIdByView`、`entryIdByFeed`、`entryIdByCategory`、`entryIdByInbox`、`entryIdByList`。时间线切换视图、切换订阅源时直接查索引，不用遍历。
- **写入走事务**：store 的每个 action 用 `createTransaction()` 同时更新内存和数据库，保证两边一致。

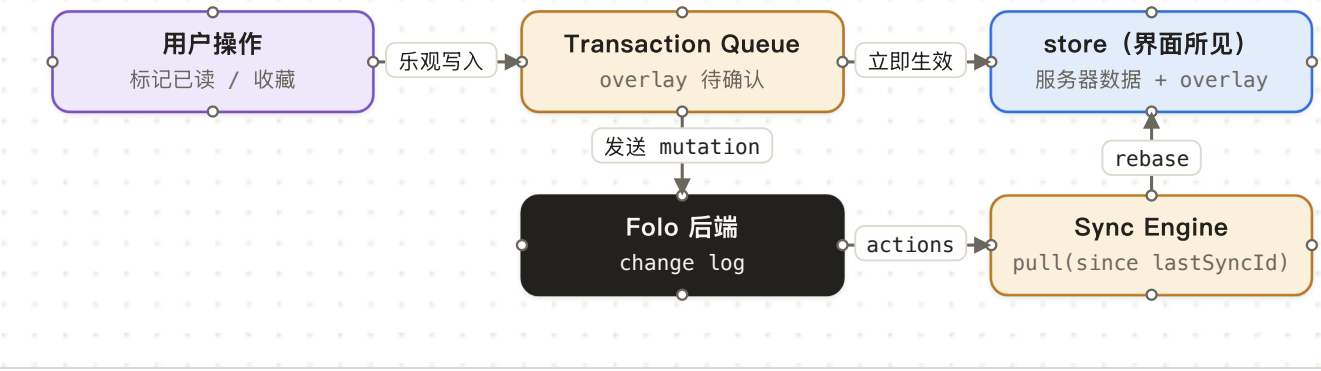
Sync Engine：模仿 Linear 的增量同步

多设备同步是 RSS 阅读器最难做好的部分：你在手机上读过的文章，电脑上要立即变成已读；离线时的操作，上线后要补发；本地的乐观更新不能被服务器的旧快照覆盖。

Folo 的方案写在 `sync-engine.ts 的注释` 里，明确说是“modelled on Linear's sync engine”：

1. 客户端记住一个游标 `lastSyncId`：已经应用过的最大变更日志 id（`class SyncEngine`）。
2. 第一次（bootstrap）从服务器取当前 id 和完整快照；之后每次只拉取“游标之后的变更”（订阅变化、已读、收藏、新条目……），逐条应用到 store。
3. 本地还没被服务器确认的操作，放在 `transaction queue` 里，以 **overlay** 的形式叠加在服务器数据之上。这样即使拉到了较旧的快照，界面也不会“回退”你的操作。
4. 拉取在启动、网络恢复、窗口回到前台、定时器、服务器 ack 时触发（`SyncPullReason`）。

增量同步与本地乐观更新

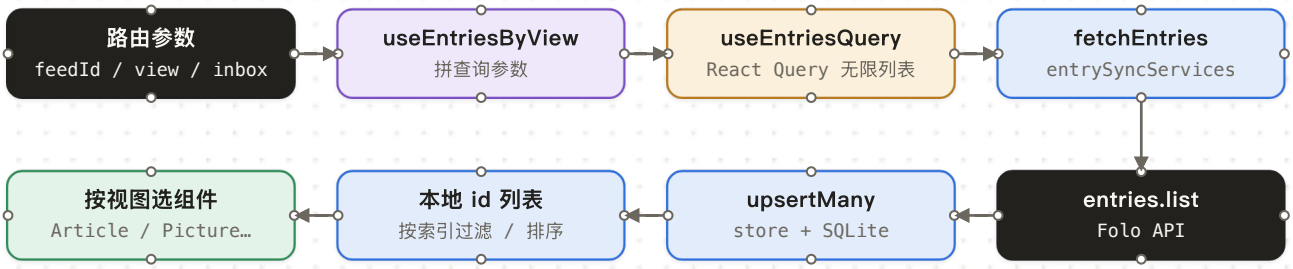


服务器确认后，overlay 被移除，store 里留下的就是服务器的真实状态。

一条时间线是怎么显示出来的

以桌面端打开“文章”视图为例：

从路由参数到列表项组件

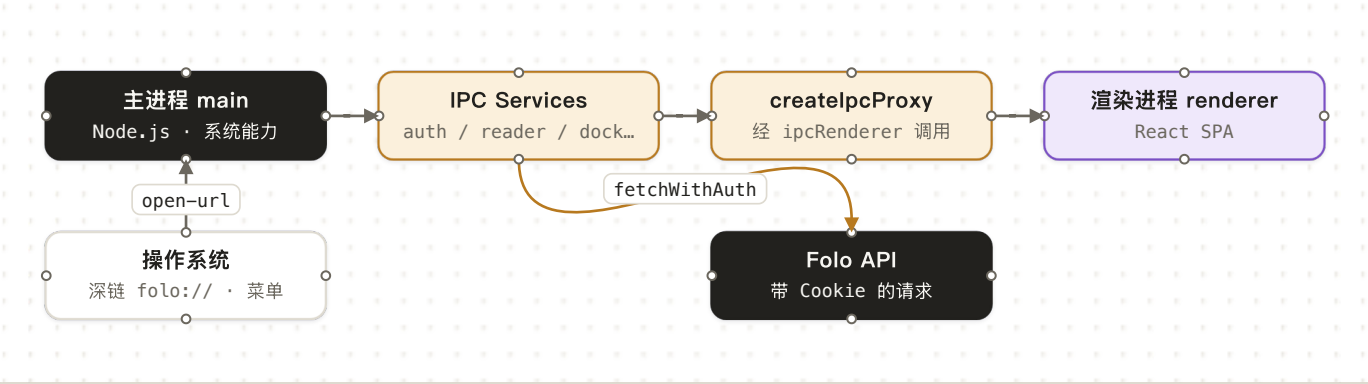


`useEntriesByView` → `useEntriesQuery` → `fetchEntries` → `getItemComponentByView`。

- **6 种视图**是 Folo 和传统阅读器最大的区别：`entry-column/Items/` 里 `article-item`、`social-media-item`、`picture-item`、`video-item`、`audio-item`、`notification-item` 各自一个组件。RSSHub 路由声明的 `view` 正是为这 6 种视图准备的元数据。
- **条目详情**在 `entry-content/`，阅读模式（readability）会请求服务端抽取正文，再由 `packages/readability` 清洗。
- **AI 排序**只是 `entries.list` 的一个 `aiSort: true` 参数（`L74`、`L534`），排序在服务端完成，结果按顺序返回一页 100 条。

Electron：主进程、渲染进程与 IPC

桌面端的两个进程



服务清单在 `ipc/index.ts`，渲染进程侧的代理在 `lib/client.ts`。

- **IPC 是类型安全的**：主进程用 `electron-ipc-decorator` 定义 `AppService`、`AuthService`、`ReaderService` 等服务类，导出 `IpcServices` 类型；渲染进程 `createIpcProxy<IpcServices>()` 得到一个“像本地对象一样调用”的代理。加一个系统能力 = 在主进程加一个方法，渲染进程就有类型提示。
- **登录态在主进程**：Electron 里的 API 请求经 `api-client.ts` 的 `fetchWithElectronAuth` 交给主进程带上 Cookie 发出。
- **深链协议**：`registerProtocols` 注册 `folo://`（开发版是 `folo-dev://`，见 `constants.ts`）。浏览器里完成 OAuth 登录后，通过 `folo://auth?token=...` 回到 App。我们本地测试时遇到的“登录跳去了官方 App”，就是因为开发版只认 `folo-dev://`；登录页底部的“输入授权令牌”就是为这种情况准备的。

AI 功能是怎么接上的

功能	客户端调用	源码
AI 对话	AI SDK 的 `HttpChatTransport` → `POST /ai/chat`，服务端流式返回 UI message chunks，工具调用（查条目、画图表…）在服务端执行	`transport.ts`
摘要	`api().ai.summary({ id, language, target })`	`summary/store.ts`
翻译	`api().ai.translationBatch()`，批量 + NDJSON 流式返回	`translation/store.ts`
AI 排序	`entries.list({ aiSort: true })`	`entry/store.ts`
付费限制	哪些设置需要付费计划	`settings/constants.ts`

要点 · 定制案例：我们在本地做的“自带 OpenAI key”

因为 AI 全在服务端，想用自己的模型就得在客户端“截流”。我们在你的 fork 里做的改动正好是一个完整的定制样板，涉及的层次也就是本章讲过的这些：

1. 在 `packages/internal/store` 新增 `local-ai` 模块（纯 `fetch` 调 OpenAI 兼容接口），由 app 注册配置；
2. 在 summary / translation / entry 三个 store 的“调用 API 之前”分支到本地实现；
3. 用 AI SDK 写了一个本地 `ChatTransport`，把当前文章 / 订阅源作为上下文，并提供 `read_entry`、`search_entries` 等本地工具；
4. 新增设置页组件，配置只存本机 localStorage，不进入会同步到服务器的设置。

这个模式（**在 store 层的 API 调用处加一个可插拔的 provider**）可以复用到很多定制上。

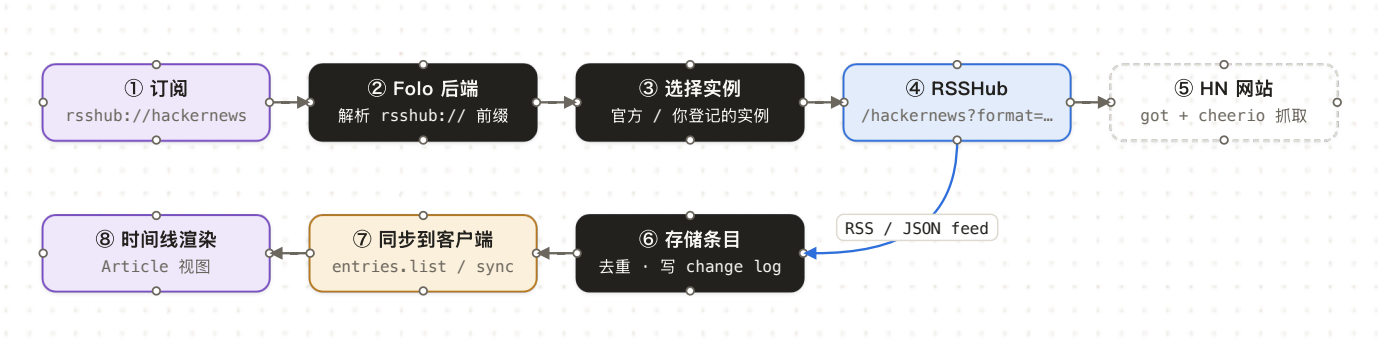
其他端：移动端、CLI、分享页

- **移动端** `apps/mobile`：Expo，复用 `packages/internal` 的 store 和 database（expo-sqlite 驱动），UI 单独实现。
- **CLI** `apps/cli`：直接用 client-sdk 调 API，输出稳定的 JSON 信封 `{ok, data, error}`，`skill.md` 就是写给 AI agent 的使用说明，我们已经把它装成了 Claude Code skill。
- **分享页** `apps/ssr`：为分享出去的订阅源 / 列表生成带 OG 标签的页面。

端到端：一条 rsshub:// 订阅的旅程

把前面所有组件串起来。假设你在 Folo 里订阅了 `rsshub://hackernews`：

从订阅到时间线



①⑦⑧ 在开源客户端里；②③⑥ 在闭源后端里，客户端只能看到相关的 API（例如登记实例的 `rsshub.use / create`）；④⑤ 在开源的 RSSHub 里。

几个要点：

- `rsshub://` **是一个“逻辑地址”**：订阅里存的不是某台具体服务器的 URL，而是路由路径。具体由哪台 RSSHub 实例去抓，由后端决定：默认由 Folo 官方的实例处理，你也可以在设置里登记自己的实例（客户端在 `modules/rsshub/`，地址校验规则见 `instance-url.ts`）。这样同一个订阅可以在不同实例之间迁移，不需要改订阅本身。
- **实例的元信息**来自 RSSHub 那边的 `/api/follow/config`：实例所有者、价格、用户上限、缓存时间、git 版本。
- **抓取频率**受两层缓存叠加影响：RSSHub 的路由级缓存（默认 5 分钟）和 Folo 后端自己的刷新策略。所以“网站刚更新、Folo 里还没出现”是正常的。
- **视图**：RSSHub 路由声明的 `view`（`ViewType`）和 Folo 的视图编号一一对应，用来提示这个源适合用哪种界面展示；订阅时你仍然可以自己选。

> **类比 · 订阅地址像收件人姓名**
>
> `rsshub://hackernews` 像快递单上的“收件人姓名”，而不是“具体哪个快递站”。Folo 后端是调度中心，决定这次交给哪个快递站（RSSHub 实例）去取件；你可以自己开一个快递站，登记到调度中心。

开源许可证：你能做什么、必须做什么

> 以下是对许可证文本的技术性解读，帮助你判断常见场景，**不是法律意见**。涉及商业发布时请咨询律师。

两个项目的许可证

项目 / 部分	许可证	依据
RSSHub	**AGPL-3.0**	`LICENSE`、`package.json` 的 `"license": "AGPL-3.0"`
Folo 客户端（整个仓库默认）	**AGPL-3.0-only**	`LICENSE`、根 `package.json`
Folo 的 `icons/mgc` 目录	**不可再分发**（版权属于 MingCute）	`README 的附加例外`、`icons/mgc/`（232 个文件）
`packages/internal/utils`、`apps/mobile/native`	MIT（package.json 声明）	各自的 `package.json`
`apps/desktop/layer/main`	package.json 写的是 GPL-3.0-only	与仓库 LICENSE 不一致，以仓库根 LICENSE 为准更稳妥
`@follow-app/client-sdk`（npm 依赖）	MIT	npm 元数据
Folo 后端	不公开	—

> **许可证变更的时间点**
>
> RSSHub 从 2018 年起一直是 MIT 许可（LICENSE 初版），在 **2025-12-28** 的提交 0db4c9ce 改为 AGPL-3.0。许可证不能追溯收回：这之前发布的版本仍然可以按 MIT 使用，但之后的代码（包括你现在 clone 到的版本和你新提交的路由）都是 AGPL-3.0。网上很多老教程还写着 MIT，就是这个原因。

AGPL-3.0 的核心规则

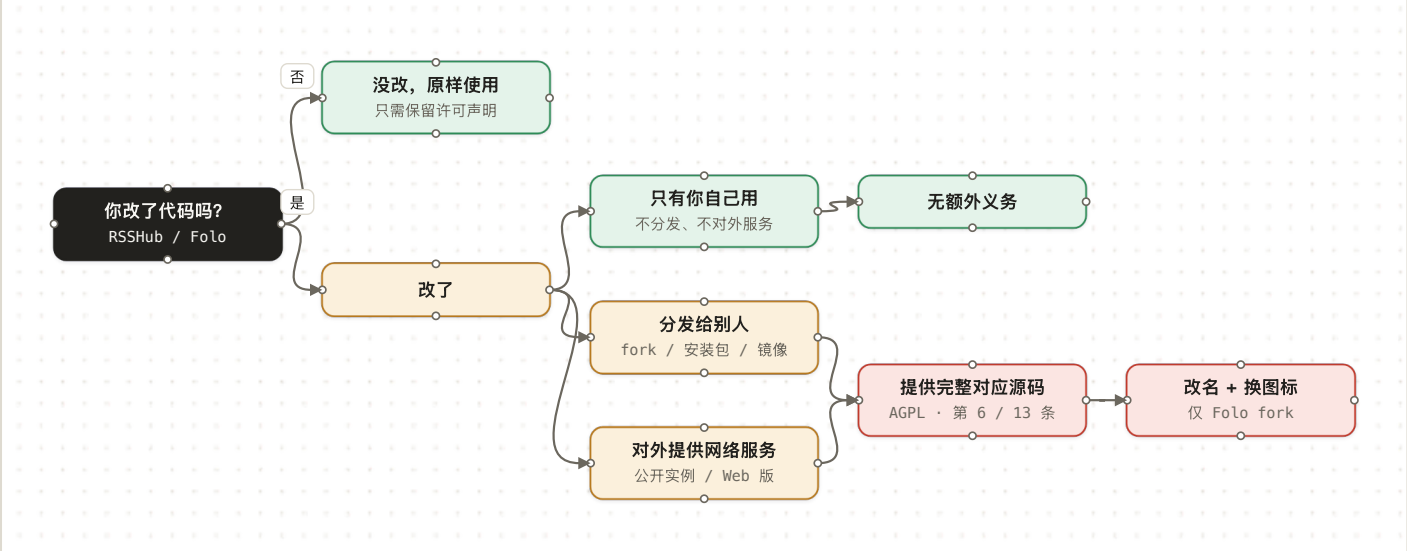
AGPL 是 GPL-3.0 加上一条“网络条款”：

1. **你可以**：自由使用、学习、修改、再分发，包括商业用途。没有“非商用”限制。
2. **分发时**（发布源码 fork、发安装包、发 Docker 镜像）：必须以同样的 AGPL 许可提供**对应的完整源代码**，保留版权和许可证声明，并标明你做了修改。
3. **网络条款（第 13 条）**：如果你**修改了**程序，并让其他人**通过网络与它交互**（比如对外开放一个改过的 RSSHub 实例，或部署一个改过的 Folo Web 版），即使你没有“分发”任何文件，也必须向这些用户提供修改后的源代码。这是

AGPL 和 GPL 的关键区别。

4. **衍生作品整体适用**：和 AGPL 代码组合成一个程序的部分（比如你加进 RSSHub 的新路由），整体要以 AGPL 发布。
5. **不授予商标权**：许可证管代码，不管名字和 Logo。

许可证义务速查：从你的使用方式出发



AGPL 的义务由“分发”或“通过网络让别人使用改过的程序”触发；只是自己改着用不触发任何义务。

常见场景对照

你想做的事	允许吗	需要做什么
本地跑、读源码、自己改着用（不给别人用）	允许	不触发任何义务
给上游提 PR	允许	你的贡献按项目许可证（AGPL）进入上游；两个项目目前**都不要求签 CLA**
自建 RSSHub 实例**只给自己**用	允许	无
自建**改过的** RSSHub 实例并**公开**给他人用 / 登记到 Folo 实例市场	允许	按第 13 条向用户提供你修改后的源码（最简单：公开你的 fork，并在实例首页给出链接）
在 RSSHub 里用 `USER_ROUTES_PATH` 加载私有路由并对外服务	允许	私有路由和 RSSHub 组合运行，对外服务时稳妥的做法是同样公开；只自用则无义务
在 GitHub 上公开你的 Folo fork	允许	保持 AGPL；**注意 `icons/mgc` 不可再分发**，严格来说应替换或移除这些图标
发布你自己编译的 Folo 安装包给别人	允许	提供对应源码；替换 `icons/mgc`；**改名、换 Logo**，避免让人以为是官方 Folo；遵守 Folo 服务条款（你的客户端仍连 `api.folo.is`）
上架 App Store	有风险	GPL 系许可证与 App Store 的使用条款存在已知冲突，个人 fork 上架需要谨慎评估
把 RSSHub / Folo 代码拿进闭源商业产品	不允许（除非整体开源）	只能以 AGPL 开源整个衍生作品

注意 · 对你的具体建议

- **只自用**：随便改，什么都不用做。
- **准备公开 fork**：先把 `icons/mgc` 换成可再分发的图标（比如其他 MIT 图标集），给项目起个新名字，在 README 写清楚“基于 Folo（AGPL-3.0）修改”。
- **对外提供服务**：在页面上放“源代码”链接指向你的 fork。
- **客户端仍连官方后端时**：记得这是在使用 Folo 的服务，要遵守它的条款，例如不要用修改过的客户端绕过服务端的付费功能。我们本地做的“自带 key”改动用的是你自己的 OpenAI 额度，没有调用 Folo 的付费 AI 服务。

贡献与定制

给 RSSHub 贡献一条路由

```
git clone https://github.com/DIYgod/RSSHub.git && cd RSSHub
pnpm i
pnpm dev                        # http://localhost:1200
pnpm new:route --help           # 交互式脚手架：命名空间、文件名、维护者、模板（API / HTML / 浏览器
# 浏览器打开 http://localhost:1200/<namespace>/<path> 检查输出
```

脚手架是 `scripts/generate-route.mjs`。提交前对照 `AGENTS.md` 里的 review 规则，最常被指出的问题：

- `example` 必须以 `/` 开头且能访问；`categories` 只写一个；`radar.source` 不带 `https://`；
- 抓详情页一定用 `cache.tryGet()`；不要对同一段 HTML 调用两次 `load()`；
- 只用 `DataItem` 里定义的字段；
- 只有真的用了浏览器才设置 `requirePuppeteer: true`；`maintainers` 写有效的 GitHub 用户名。

> **要点 · 不想合并到上游时：私有路由**
>
> 设置 `USER_ROUTES_PATH=/app/routes-user`，在里面放独立的 `.mjs` 文件，每个文件导出 `namespace` 和 `routes`，就能注册新的命名空间，同样享受缓存、过滤参数和多格式输出（见 `lib/user-routes.ts`）。适合给公司内网、个人账号这类不适合公开的来源写 feed。

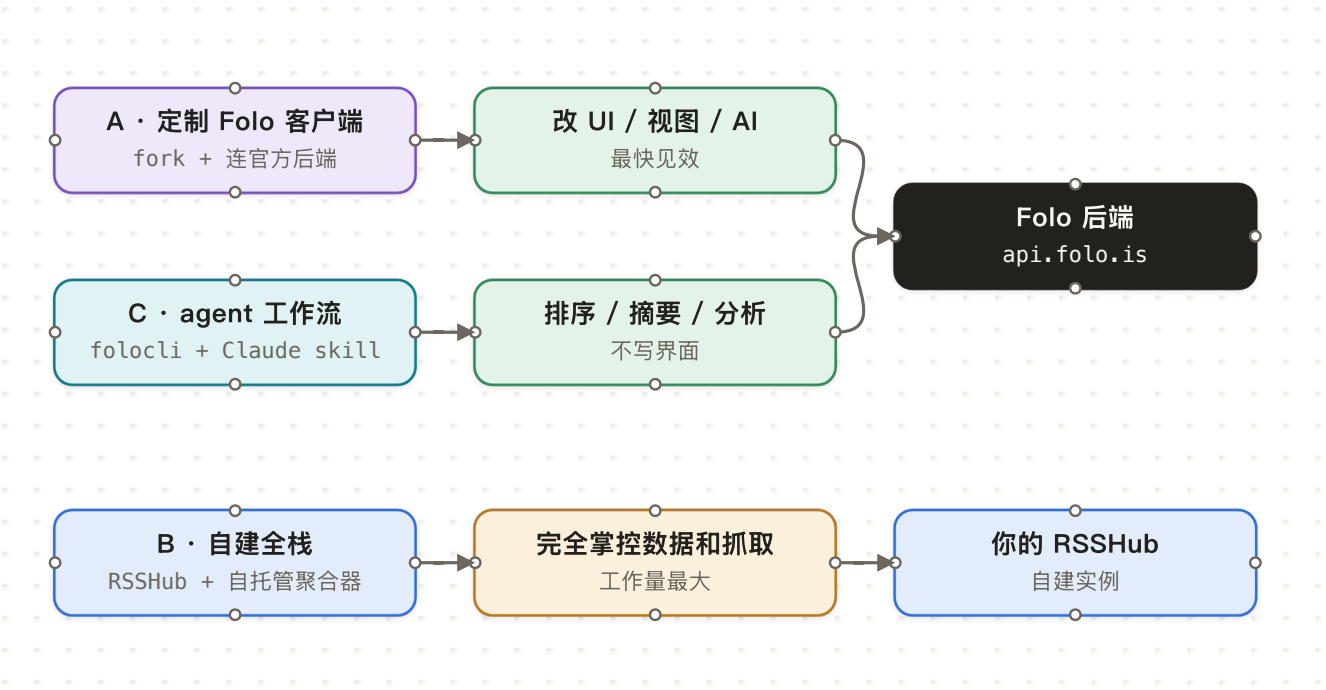
给 Folo 贡献

```
git clone https://github.com/RSSNext/Folo.git && cd Folo
corepack enable && corepack prepare
pnpm install
cd apps/desktop && pnpm run dev:web       # 浏览器开发，连接线上 API
# 或者：pnpm run dev:electron             # Electron 开发
```

- 提交前按 `AGENTS.md` 的顺序：`pnpm run typecheck` → `pnpm run lint:fix` → `pnpm run test`。
- 详细说明在 `CONTRIBUTING.md`，包括 Electron 登录问题的处理（复制 `__Secure-better-auth.session_token`）。
- **适合入手的方向**：某一种视图的列表项 / 详情布局（`Items/`）、设置页、集成（`integration.ts`：发送到 Obsidian / Readwise…）、阅读模式清洗、CLI 命令、国际化文案（`locales/`）。
- **需要谨慎的方向**：任何依赖服务端新接口的功能。你没法改后端，PR 只能用已有的 API。

打造“自己喜欢的阅读器”：三条路线

三条定制路线



路线 A、C 依赖 Folo 官方服务；路线 B 完全独立，但需要自己写或选一个后端 / 客户端。

路线	做法	优点	代价
A. 定制 Folo 客户端	fork Folo，改 UI、视图、快捷键、AI provider（参考我们做的本地 OpenAI 改动），连接官方后端	继承 Folo 全部功能和多端同步；改动可以回馈上游	受限于官方 API 和服务条款；后端行为改不了
B. 自建全栈	自建 RSSHub 实例 + 一个可自托管的聚合后端（自己写，或选用其他开源的自托管阅读器服务），再配一个客户端	数据、抓取频率、AI 全部自己掌控	Folo 客户端和它的 API 深度绑定，无法直接换后端；工作量最大
C. agent 工作流	用 folocli + Claude skill（已装好），让 AI 按你的标准排序、摘要、写日报	零界面开发，马上可用	只是“读和管理”，不是完整的阅读器界面

一个务实的组合是：**先用 C 验证你想要的“排序 / 摘要”标准，再用 A 把它做进界面；如果某个网站没有 feed，就给 RSSHub 写一条路由（或私有路由），然后在 Folo 里订阅** `rsshub://你的路由` **。**

附录：源码索引

RSSHub

主题	位置
进程入口 / 监听	`lib/index.ts`
中间件顺序	`lib/app-bootstrap.tsx`
路由表加载	`lib/registry.ts`
注册与懒加载	`lib/registry-helpers.ts`
路由表生成	`scripts/workflow/build-routes.ts`
类型契约	`Route` · `Data` · `DataItem` · `ViewType` · `RadarItem`
路由级缓存	`lib/middleware/cache.ts`
详情级缓存	`cache.tryGet`
过滤参数	`lib/middleware/parameter.ts`
输出渲染	`lib/middleware/template.tsx` · `lib/views/`
示例路由	`hackernews` · `namespace.ts`
HTTP / 浏览器	`ofetch.ts` · `playwright.ts`
Radar 规则接口	`lib/api/radar/rules/all.ts`
Folo 专用接口	`lib/api/follow/config.ts`
配置项	`lib/config.ts`
Workers 入口	`lib/worker.ts`
私有路由	`lib/user-routes.ts`

Folo

主题	位置
API 客户端	`api-client.ts`
启动流程	`initialize/index.ts`
本地库 → store	`hydrate.ts`
数据库驱动 / schema	`db.desktop.ts` · `db.rn.ts` · `schemas`
同步引擎	`sync-engine.ts` · `transaction-queue.ts`
条目 store / 拉取	`EntryState` · `fetchEntries` · `useEntriesQuery`
视图组件	`getItemComponentByView.ts`
Electron IPC	`main/ipc` · `renderer/lib/client.ts`
深链协议	`manager/app.ts` · `constants.ts`
RSSHub 实例管理	`queries/rsshub.ts` · `modules/rsshub/`
AI	`transport.ts` · `summary` · `translation`
付费设置	`settings/constants.ts`
CLI	`program.ts` · `skill.md`
许可证	`LICENSE` · `README 例外条款`